

Exploits de kernel en Windows (ultimo recurso)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un exploit de kernel de Windows aprovecha un fallo en el núcleo o en un driver para ejecutar código como SYSTEM desde un usuario normal. Aplica cuando faltan parches concretos (MS16-032, PrintNightmare, etc.). Se identifican comparando los hotfixes instalados con los CVE conocidos.

Como en Linux, se dejan para el FINAL: pueden inestabilizar o tumbar la máquina, y casi siempre hay un vector de config más limpio. Solo cuando la enumeración (enum-privesc-windows) no dio token, servicio ni credencial se recurre a esto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ningún vector de config funciona y systeminfo revela un Windows con pocos parches. Watson/wesng cruzan los hotfixes con exploits conocidos.

Cómo identificarla

- systeminfo muestra pocos hotfixes o un build viejo.
- Watson/wesng sugieren exploits aplicables a esa build.
- Servicios de impresión vulnerables (PrintNightmare) presentes.

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Pasa systeminfo por wesng/Watson y compara con los KB instalados.

Evidencia esperada: Un CVE de Elevation of Privilege sin el KB correspondiente instalado.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
systeminfo > sysinfo.txt
```

Objetivo y contexto: Captura version, build y hotfixes. Es la entrada para las herramientas que sugieren que exploit de kernel aplica a esta maquina.

Resultado esperado: OS version/build y lista de KB. Guardalo y pasalo por wesng: te dara CVEs sin parchear con exploit publico.

Capa 2: prueba documentada

```
python wesng.py sysinfo.txt
```

Objetivo y contexto: Windows Exploit Suggester cruza tu systeminfo con la base de vulnerabilidades y lista exploits candidatos ordenados por relevancia.

Resultado esperado: Lista de CVE/MS con enlace al exploit. Prioriza los de 'Elevation of Privilege' con PoC; leelo antes de ejecutar (exploitar-cve).

Anatomía de los payloads

La primera prueba es `systeminfo > sysinfo.txt`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un CVE de Elevation of Privilege sin el KB correspondiente instalado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Saca version y parches con systeminfo.
- Pasa la salida por wesng/Watson para candidatos.
- Elige uno, leelo (exploitar-cve) y consigue el binario compilado fiable.
- Ejecuta con cuidado; confirma SYSTEM. Si falla o cuelga, vuelve a los vectores de config.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El sistema esta al día de parches para los CVEs conocidos.
- Quedan vectores de configuracion sin probar (token, servicios, credenciales): prueba esos primero.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si faltan parches, un exploit local eleva a SYSTEM; deja esto para el final por inestabilidad. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y precondiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook](#) (kernel-exploits-windows)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.